

SYSTEMVERILOG Problems and Solutions

arb priority solution

```
// arb_prio4_solution.sv
module arb_prio4(input logic clk, rst_n, input logic [3:0] req, output logic [3:0] gnt);
    always_comb begin
        gnt = 4'b0;
        if (req[3]) gnt = 4'b1000;
        else if (req[2]) gnt = 4'b0100;
        else if (req[1]) gnt = 4'b0010;
        else if (req[0]) gnt = 4'b0001;
    end
endmodule
```

arb priority starter

```
// arb_priority_starter.sv - 4 port fixed priority
module arb_prio4(input logic clk, rst_n, input logic [3:0] req, output logic [3:0] gnt);
    // TODO
endmodule
```

arb round robin solution

```
// arb_round_robin_solution.sv
module arb_rr #(parameter N=4)(input logic clk, rst_n, input logic [N-1:0] req, output logic [N-1:0] gnt);
    logic [$clog2(N)-1:0] ptr;
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) ptr <= 0;
        else begin
            gnt = '0;
            for (int i=0; i<N; i++) begin
                int idx=(ptr+i)%N;
                if (req[idx]) begin gnt[idx]=1; ptr<= (idx+1)%N; break; end
            end
        end
    end
endmodule
```

arb round robin starter

```
// arb_round_robin_starter.sv - parameterizable N
module arb_rr #(parameter N=4)(input logic clk, rst_n, input logic [N-1:0] req, output logic [N-1:0] gnt);
    // TODO
endmodule
```

axi lite stub solution

```
// axi_lite_stub_solution.sv
// A full AXI-lite slave is long; here is a minimal conceptual stub illustrating handshake
module axi_lite_stub(input logic ACLK, ARESETn,
    input logic AWVALID, output logic AWREADY, input logic [31:0] AWADDR,
    input logic WVALID, output logic WREADY, input logic [31:0] WDATA,
    output logic [1:0] BRESP, output logic BVALID, input logic BREADY,
    input logic ARVALID, output logic ARREADY, input logic [31:0] ARADDR,
    output logic RVALID, input logic RREADY, output logic [31:0] RDATA);
    // Very small simplified behavior: accept handshakes immediately
```

```

    assign AWREADY = AWVALID; assign WREADY = WVALID; assign BVALID = AWVALID & WVALID;
    assign BRESP = 2'b00;
    assign ARREADY = ARVALID; assign RVALID = ARVALID; assign RDATA = 32'hDEADBEEF;
endmodule

```

axi lite stub starter

```

// axi_lite_stub_starter.sv - minimal AXI-lite slave stub
// TODO: implement minimal handshake for read/write

```

cache lru solution

```

// cache_lru_solution.sv - conceptual 4-line 2-way set-associative with LRU bits
module cache_lru(input logic clk, rst_n, input logic req, input logic [1:0] addr, input logic [31:0] wdata,
    logic [1:0] tag [0:1]; logic valid [0:1]; logic [31:0] data [0:1]; logic lru; // greatly simplified
    // On access, check way 0/1, update lru bit on misses/ways; concept only
    assign hit = (valid[0] && tag[0]==addr) || (valid[1] && tag[1]==addr);
    always_ff @(posedge clk) if (req && we) begin if (!hit) begin // refill simple
        tag[0] <= addr; valid[0] <= 1; data[0] <= wdata; lru <= ~lru; end else data[0]<=wdata; end
endmodule

```

cache lru starter

```

// cache_lru_starter.sv - small 4-line cache LRU policy
module cache_lru(input logic clk, rst_n, input logic req, input logic [1:0] addr, input logic [31:0] wdata,
    // TODO implement small direct-mapped or set-associative with LRU
endmodule

```

dual port ram solution

```

// dpram_solution.sv
module dpram #(parameter W=8, DEP=256) (input logic clk, we_a, we_b, input logic [$clog2(DEP)-1:0] addr_a,
    logic [W-1:0] mem [0:DEP-1];
    always_ff @(posedge clk) begin
        if (we_a) mem[addr_a] <= din_a;
        dout_a <= mem[addr_a];
        if (we_b) mem[addr_b] <= din_b;
        dout_b <= mem[addr_b];
    end
endmodule

```

dual port ram starter

```

// dual_port_ram_starter.sv
module dpram #(parameter W=8, DEP=256) (input logic clk, we_a, we_b, input logic [$clog2(DEP)-1:0] addr_a,
    // TODO implement sync dual-port RAM
endmodule

```

fifo param solution

```

// fifo_param_solution.sv
module fifo_param #(parameter W=8, DEP=16) (input logic clk, rst_n, input logic wr, rd, input logic [W-1:0] data,
    logic [W-1:0] mem [0:DEP-1]; logic [$clog2(DEP)-1:0] head, tail; integer count;
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin head<=0; tail<=0; count<=0; end else begin
            if (wr && count<DEP) begin mem[head]<=data; head<=head+1; count<=count+1; end
            if (rd && count>0) begin dout<=mem[tail]; tail<=tail+1; count<=count-1; end
        end
    end
endmodule

```

```

        end
    end
    assign empty = (count==0); assign full = (count==DEP);
    assign almost_full = (count >= DEP-1); assign almost_empty = (count <= 1);
endmodule

```

fifo param starter

```

// fifo_param_starter.sv - parameterizable FIFO with almost_full/empty
module fifo_param #(parameter W=8, DEP=16) (input logic clk, rst_n, input logic wr, rd, input logic [W-1:0] din, output logic [W-1:0] dout);
    // TODO
endmodule

```

fifo simple solution

```

// fifo4_solution.sv
module fifo4 #(parameter W=8) (input logic clk, rst_n, input logic wr, rd, input logic [W-1:0] din, output logic [W-1:0] dout);
    logic [W-1:0] mem [0:3];
    logic [1:0] head, tail; logic [2:0] count;
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin head<=0; tail<=0; count<=0; end else begin
            if (wr && !full) begin mem[head]<=din; head<=head+1; count<=count+1; end
            if (rd && !empty) begin dout<=mem[tail]; tail<=tail+1; count<=count-1; end
        end
    end
    assign empty = (count==0); assign full = (count==4);
endmodule

```

fifo simple starter

```

// fifo_simple_starter.sv - depth 4
module fifo4 #(parameter W=8) (input logic clk, rst_n, input logic wr, rd, input logic [W-1:0] din, output logic [W-1:0] dout);
    // TODO
endmodule

```

fsm4 solution

```

// fsm4_solution.sv
module fsm4(input logic clk, rst_n, input logic start, input logic done, output logic [1:0] state);
    typedef enum logic [1:0] {IDLE=0, READ=1, WRITE=2, DONE=3} st_t;
    st_t cur, nxt;
    always_ff @(posedge clk or negedge rst_n) if (!rst_n) cur<=IDLE; else cur<=nxt;
    always_comb begin nxt = cur; case(cur)
        IDLE: if (start) nxt = READ;
        READ: if (done) nxt = WRITE;
        WRITE: if (done) nxt = DONE;
        DONE: nxt = IDLE;
    endcase end
    assign state = cur;
endmodule

```

fsm4 starter

```

// fsm4_starter.sv
// States: IDLE=0, READ=1, WRITE=2, DONE=3
module fsm4(input logic clk, rst_n, input logic start, input logic done, output logic [1:0] state);
    // TODO
endmodule

```

handshake solution

```
// handshake_solution.sv
module handshake(input logic clk, rst_n, input logic req, output logic ack);
    logic pending;
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) pending <= 1'b0;
        else begin
            if (req) pending <= 1'b1;
            else if (ack) pending <= 1'b0;
        end
    end
    assign ack = pending & ~req; // simplistic example
endmodule
```

handshake starter

```
// handshake_starter.sv
module handshake(input logic clk, rst_n, input logic req, output logic ack);
    // TODO: implement two-cycle handshake
endmodule
```

rr arbiter2 solution

```
// rr_arbiter2_solution.sv
module rr_arbiter2 (input logic clk, rst_n, input logic req0, req1, output logic gnt0, gnt1);
    logic ptr;
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) ptr <= 0;
        else if (gnt0) ptr <= 1;
        else if (gnt1) ptr <= 0;
    end
    always_comb begin
        gnt0 = 0; gnt1 = 0;
        if (req0 && req1) begin if (ptr==0) gnt0=1; else gnt1=1; end
        else if (req0) gnt0=1; else if (req1) gnt1=1;
    end
endmodule
```

rr arbiter2 starter

```
// rr_arbiter2_starter.sv
module rr_arbiter2 (input logic clk, rst_n, input logic req0, req1, output logic gnt0, gnt1);
    // TODO: implement
endmodule
```